Day2

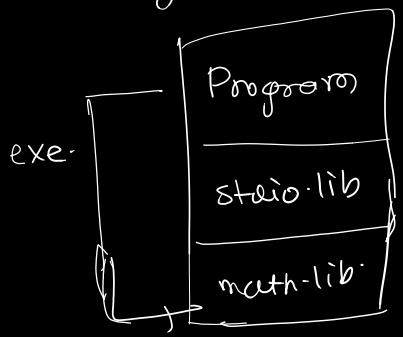
Libraries.

Binary formats.

① SLL → Static Linked Library.

② DLL → Dynamic Linked Library.

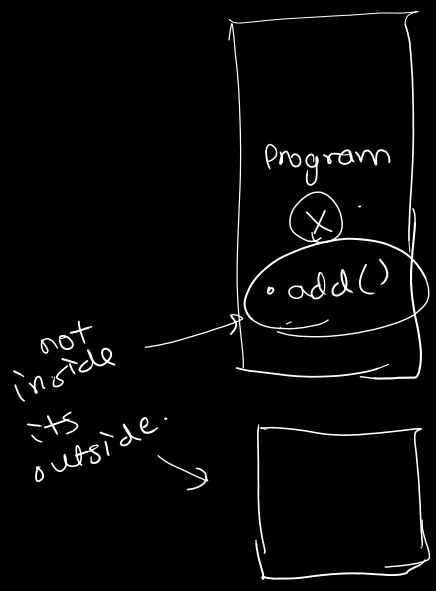
carrier is → exe.



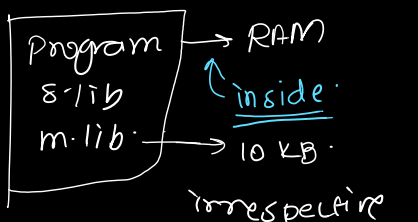
(static)

Independently used.

but libraries are always there.

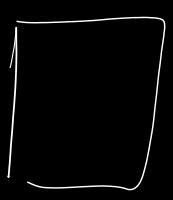


Static :- Program & lib are linked at the time of linking itself.



Irrespective we are using it or not.

Dynamic — only program in Ram.



away from program.exe.

OS

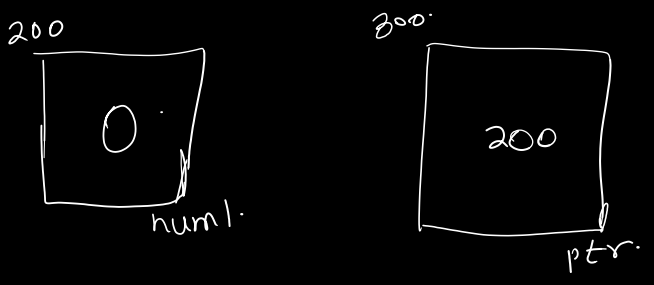
Implicity :- SLL + DLL

* Static → Linkers.

Dynamic → at Runtime.

* Pointers :-

We use it to pass the value which is live but its slope is not there, where we want to access it.



*ptr = 10 — This is pointer.

* Different types of Programming Languages.

1. Procedural based :- C → methods, functions by function execution.
2. Object Based language :- Basic
3. Object Oriented language :- Java, C++, python, .NET → Real-world entity.

* object → a) Identity
b) State or Properties.
c) Behaviours / Methods.

*

Major → Language have to be features.

1. Abstraction. (or Data Abstraction?)
2. Encapsulation / Hidding). (or Data Encapsulation)
3. Inheritance.
4. Polymorphism. (Modularity)
5. Modularity.

Minor

1. Persistance/serialization.
2. Concurrency.
3. RTTI / RTCI
   - Runtime Type Identification or Runtime Class Information.

(*) **Abstraction** :- Generalisation.

Abstraction is also called as generalisation.

What are we hiding specific to both those entities which is in common.

```
class sedan {
        Engine cc
        Wheels.
}
```

```
Functions.
startengine(){
}
—//—
```

Please create another class representing SUV.

```
class SUV {
    same as sedan
}
```

```
Functions(){
They will be also same.
}
```

What we are doing?:- We are repeating the code again and again for every type.

All properties common of both.

So what we are doing, we will put the common things in higher class called Passenger car.

```
class Passengercar{
}
```

//This is abstraction class.

Abstraction is a feature that allows you to gather common features from one or more entity to form real-time entity.

* Encapsulation :- Human Body

Definition :-

* Binding of data and behaviour into a single unit is called encapsulation.

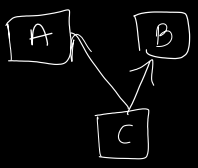
* Inheritance :- reverse of abstraction :- Specialisation.

Forming a specialised Form from common features.

⇒ Specialised feature (new) + Common Features.

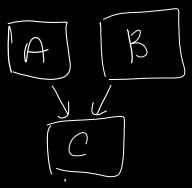
* Inheritance :-
types
a) single level
b) multiple
c) Hybrid
d) Hierarchical.

B is inherited from A.

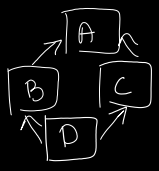


multiple inheritance

multilevel inheritance.



single inheritance.



Doubt of student

* ClassA {
int data;
void display;
}

class B {
int data;
int num;
void show() {
}
}

* ④ Polymorphism :- Ability of an object to behave differently based on the different kinds of inputs.

CompileTime Polymorphism
Static
Early binding

RunTime Polymorphism.
Dynamic
Late binding

Function overloading:- You have multiple functions with same name:-

But :- ① either the type of parameter change.

② No. of parameter should change.

③ The sequence of datatype of function should change.

e.g.

* add (int, float).
* add (float, int).
*

④ Function overloading does not happen between functions with different overloading.

eg. int add (int).
float add (int).

C :- ✗ function overloading.

* printf ("%d", num).
* printf ("%f", value)

*

printf; ⇒ Function overloading in C or not?

⇒ No. But printf?

↓

int printf (const char*, ...);

↑ ellipses.

variable argument lists.

* Name mangling :-

(Done by compiler.)

↓

Only occurs on Function name.

C++ :- Function name + Parameters of function.

How function overloading occurs

add @ dlfl
add @ fldl. } Name Mangling.

↓

Considers both Function Name and Parameters that's why function overloading occurs in C++.

* Types of Polymorphism

↓ Static

↓ Dynamic.

→ We do it only in Inheritance.

* Marriage

↓

Parents (Arranged marriage).

↓ Static

↓ Dynamic.

Whatever Parent doing arranged, child is doing.

Even if Parent arranged, but option to go love as well. (flexibility.)

⇒ which means are allowing to modify by child.

Parent.

Forces.

Arranged marriages

But ran away for love.

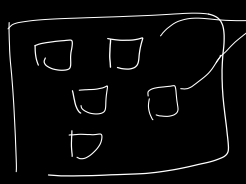
← Even if Restriction from Parent, child still modifies.

* Whenever want to change default behaviour, we go to dynamic.

* Modularity. :-
  * Reduces Complexity.
  * Easy to maintain.
  *

a) Monolithic App^{n}. → Single unit of application.

b) Component based App^{n}. →



→ different components in single unit.

Advantages:- ① Same component can be used in app1, 2, 3, 4, 5 ....

② Dynamically u can add and remove any component.

* Modularity is a feature to breakdown code into multiple modules which are independent to each other but they work in association.

⊛ Minor Features:-

① Persistance or Serialization:-

Persistance is the ability of maintaining current state.

Student → name. } Persistance.

rollno, course

(Store data store kiya.)

1, Pritish, Java, 99999.

e.g. .csv file (something)

rollno =

name =

← meta information (Serialization.)

With meta information. + data

e.g. xml + tags.

* Concurrency ⇒ Ability of any object, to perform multiple task parallelly

↓

Achieved using multi-threading

* RTTI ⇒ This feature to identify the type of an object at runtime

↓

We require it at generalisation

Flying object and Parrot example by Sir.

*

iska bada bhai (advancement of RTTI)

RTCI ⇒ Not only object, it gives all information of the class and everything inside it.

In C++ :- What feature we cant do ?:-

* Concurrency ? ✓

In C :- Abstraction, Encap, Inheritance, Polymorphism

Partially. cannot do.

Concurrency can be done in C.

Windows made using :- C

What is multithreading ?

*

C → process.h

⇒ _beginthreadex.

explore...

* object based language.

vs

object oriented language.

↓

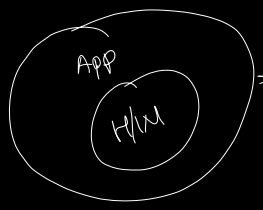
Object based language :- does not support inheritance.

↓

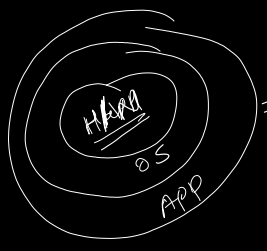
And so would be only supporting static polymorphism.

So hence can't say its an oops language.

* Topic :-

① 

⇒ Embeaded system. ⇒ like washing machine.

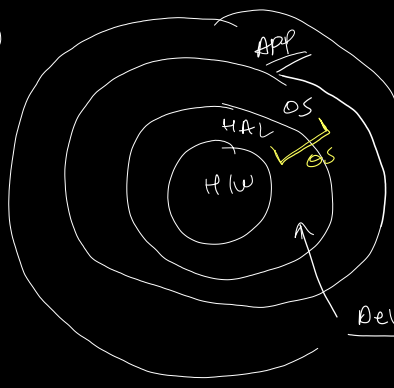
② 

⇒ Laptops.

Better Performing is ① . Because it directly communicates with hardware.

* But Problem with ① .
  * It got dependency on hardware.
  * It should know the working of the hardware.

*

2a 

HAL :- Hardware abstraction layer.

HAL + OS ↓ work together.

*
COFF → only windows.
hence bin can't run on windows and exe file can't run on linux.

exe → binary
bin → binary
still diff they diff because of diff format.

* Windows ⇒ CreateWindow(1);
  Linux ⇒ openWindow(3);
  } ①② communication style is different

API.

Problems observed by James Gosling

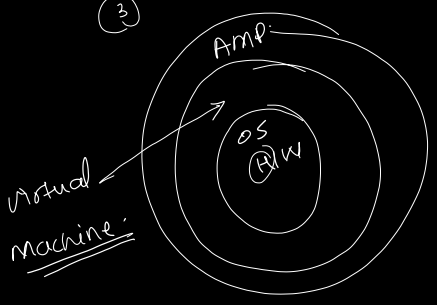
Hence ; 1) format
2) communication
prevents them to be used switching into one another.

↓
Repetitive working codes we working because of this. ⇒ By year 2000, Every American home, there will use microprocessor.

* He said let me create ~ * Language which can be platform independent!

*

(3)



Virtual m/c is solving the problem of Platform dependency.

* Communication will be lengthy. now in this care. ⇒ So he made the first language called as OAK.

OAK. → changed to → Java. (officially introduced in 1994.)

Java ← First Objected Oriented Language

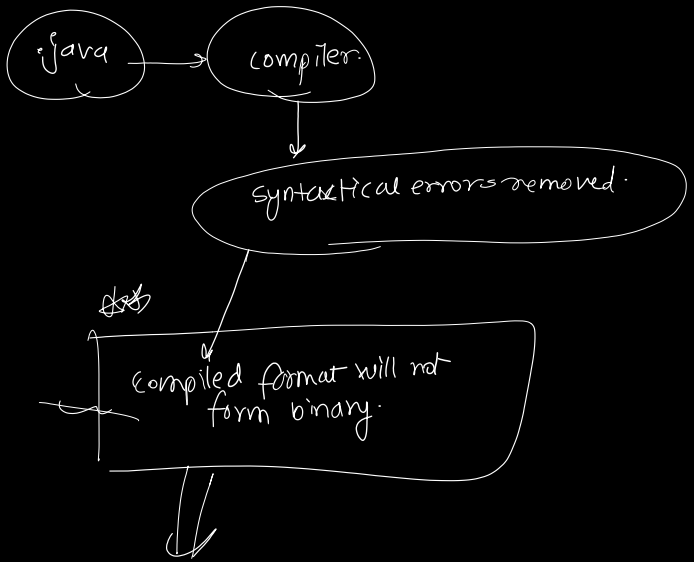
So to avoid effort and concepts. ← he used syntax from C and OOP from C++.

But objective was to make this language platform independent.

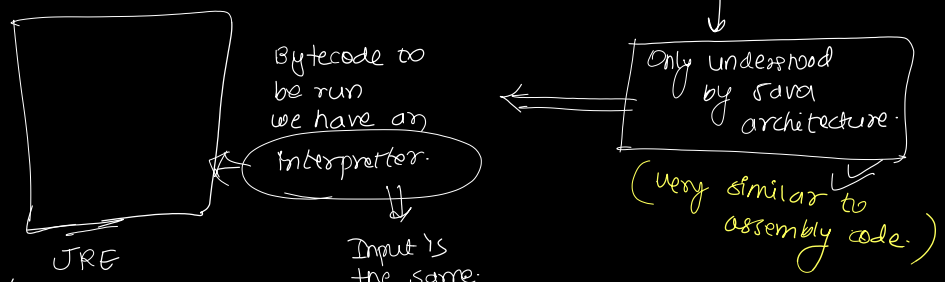
Platform Independent? How did he made.

- Compiled e.g. (C++).
- Interpreted. e.g. (Basic) (Read & execute Line by Line.)
- Pased language) (using symbol.)

Java was partly compiled partly interpretted.



Instead a bytecode is formed.



(1) (2)

18:45

(*)

(1) J2SE (2) J2EE (3) J2ME.

New J SE
Java standard edition.

Java enterprise edition.
⇓
Used for distributed architecture

* Like Amazon.

*

Java micro edition.
⇓
PDA, mobile, Blu Ray,

J.D.K. CDAC upgraded to partly 17.0.

8.0 & 11.0 ⇒ totally used in industry

* Java SE.

Break up
at 6:57. to 7:10
write from memory.

```
class Program
{
    public static void main (String[]args)
    {
        System.out.println ("Hello World").
    }
}
```

```
javac -verson.
javac Anything.java
```

Download Java.